Кейс-чемпионат
Своя База

Подготовили:
Шубенина Д.В.
Рыжкин А.А.

Оглавление

Блок 1. Зачем нужны базы данных

Блок 2. Основы реляционной модели и SQL

Блок 3. Основы взаимодействия с СУБД

Блок 4. План кейс-чемпионата

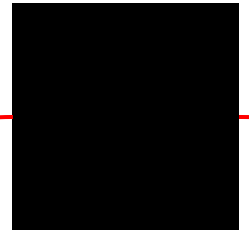
Копия презентации в папке (DB_Pres.pdf)

Зачем нужны базы данных

Проблема

- Хотим написать свой интернет-магазин – нужно как-то **сохранять** зарегистрированных пользователей
- Идея: записывать всех пользователей в массив
- Закрыли программу – все пропало

В поисках постоянного хранилища



Файл

Текстовые

Двоичные

```
{
  "users": [
    {
      "age": 16,
      "name": "Вася”
    }
  ]
}
```

JSON

```
age;name
16;Вася
67;Юля
```

CSV

```
<?xml version="1.0" encoding="UTF-8"?>
<users>
  <user>
    <age>16</age>
    <name>Вася</name>
  </user
</users>
```

XML

```
00110001 00110110
00111011 11010000
10010010 11010000
10110000 11010001
10000001 11010001
10001111
```

База данных

База данных – это организованная совокупность связанных между собой данных, хранящихся в электронном виде.

connections.txt

users.order_id ⟶ orders.id
orders.product_id ⟶ products.id

Связи между данными зафиксированы

users

orders

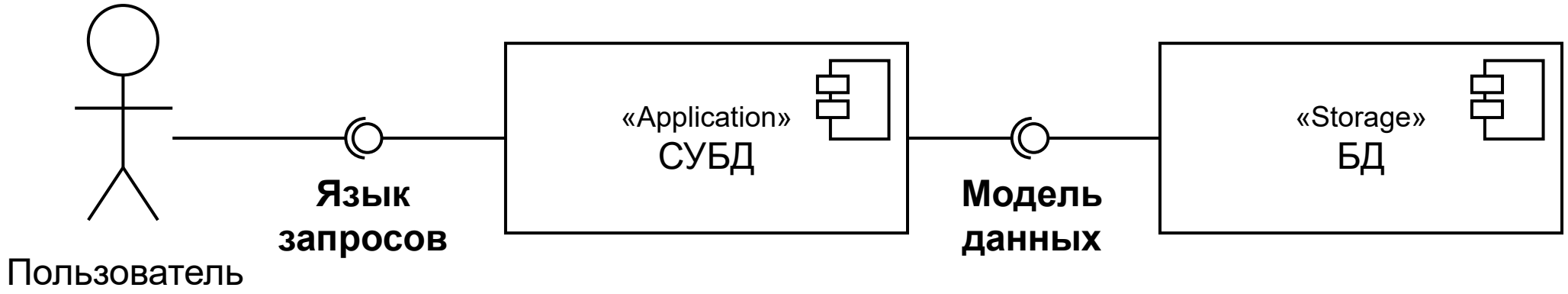
products

Файлы данных имеют определенную структуру

– это база данных (пусть и простая)

Система управления БД (СУБД)

СУБД – это программный комплекс, выступающий посредником между пользователями и данными.

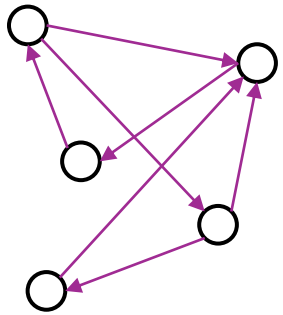


СУБД обеспечивает выполнение 4 основных операций с данными: создание (**C**reate), чтение (**R**ead), обновление (**U**pdate) и удаление (**D**elete).

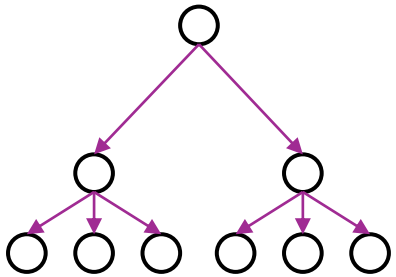
Основные модели данных

В основе каждой базы лежит модель данных – она определяет устройство хранилища и способы связи его элементов

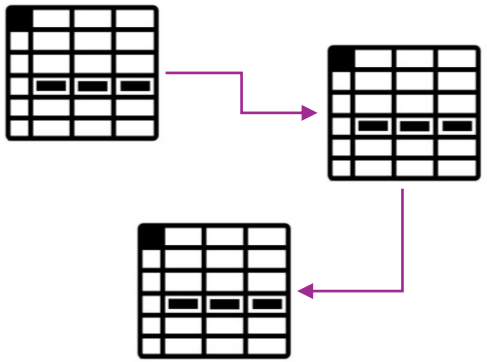
Сетевая



Иерархическая



Реляционная



Основы реляционной модели и SQL

Реляционная модель данных

- Данные хранятся в таблицах – отношениях
- Для работы с данными определен набор операций – реляционная алгебра
- На данные накладываются жесткие требования (**ограничения**) **целостности**:
  - **первичный ключ** – уникальный идентификатор каждой строки,
  - **внешние ключи** определяют связи между отношениями,
  - **другие ограничения** (уникальности, правильности значений и т.д.)
- У всего есть явная **схема** – принцип хранения данных

Отношения и связи между ними

Users

ID	Имя	Возраст	Дата рождения	ID заказа
1	Вася	16	20.04.2010	2
2	Юля	67	17.04.1959	1

схема отношения
ID, Имя, Возраст, Дата рождения, ID заказа

– строка отношения (кортеж)

– значение атрибута «Возраст» строки 2

Orders

ID	Статус	Дата создания
1	Собран	13.03.2026
2	Отменен	06.01.2025

– атрибут отношения

Первичный ключ Users – **ID**
Внешний ключ Users – **ID заказа**

Схема базы данных – **[Users, Orders]**

Язык запросов

- Операции реляционной алгебры реализуются через язык SQL (Standard Query Language)
- Операторы SQL складываются в **предложения**, которые разделяются символом «**;**»
- Операторы SQL реализуются путем написания алгоритмов на языках более низкого уровня (C/C++, Python, Rust, и т.д.)

DDL – Data Definition Language	**DML – Data Manipulation Language**
Язык определения данных – используется для создания/изменения/удаления таблиц, ограничений целостности, баз данных и т.д.	Язык манипуляции данными – используется для выполнения CRUD-операций над данными в таблицах.

DDL

Создание

```
CREATE [IF NOT EXISTS] TABLE users (
  id       INTEGER     PRIMARY KEY,
  name     VARCHAR(50) NOT NULL,
  order_id INTEGER,
  FOREIGN KEY (order_id) REFERENCES orders(id)
);
```

- **имя нового отношения**
- **атрибуты**
- **типы атрибутов**
- **ограничения целостности**, могут быть заданы при объявлении атрибута столбца или после, через запятую

Изменение

```
ALTER TABLE users
  ADD COLUMN age INTEGER CHECK (age >= 0);
```

- **имя изменяемого отношения**
- **атрибуты**
- **типы атрибутов**
- **ограничения целостности**

Удаление

```
DROP TABLE users;
```

- **имя удаляемого отношения**

DML

Выборка

```
SELECT name, age FROM users WHERE id = 1;
```

- **проекция** – атрибуты, которые хотим вывести (если хотим все, то вместо перечисления пишем «*»)
- **отношение**, из которого делаем выборку
- **фильтр** – логическое выражение, ограничивающее выборку

Вставка

```
INSERT INTO users VALUES (1, ‘Вася’, 16);
```

- **отношение**, в которое вставляем строку(и)
- **кортеж(и) значений** – кортеж(и), содержащий(е) значения атрибутов схемы отношения

Обновление

```
UPDATE users SET age = 21 WHERE id = 1;
```

- **отношение** строку(и) которого обновляем
- **атрибут(ы) и новое(ые) значения**
- **фильтр** – логическое выражение, ограничивающее строки, для которых обновим значение атрибута(ов)

Удаление

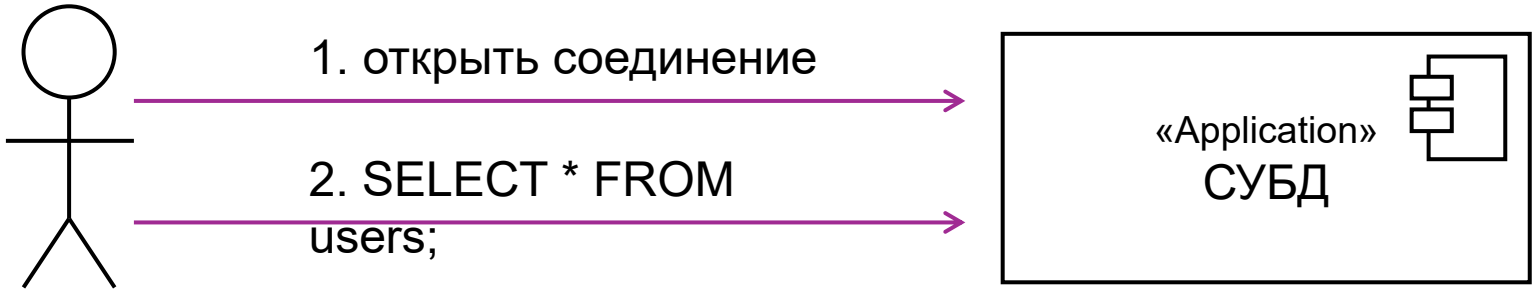
```
DELETE FROM users WHERE id = 1;
```

- **отношение**, из которого удаляем строки
- **фильтр** – логическое выражение, ограничивающее строки, которые будут удалены

Основы взаимодействия с СУБД

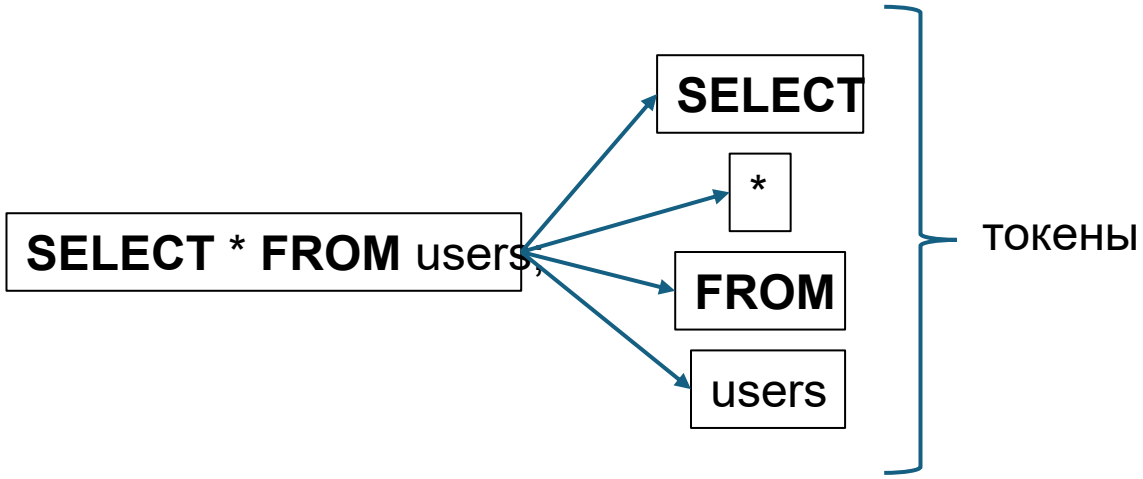
Жизненный цикл запроса

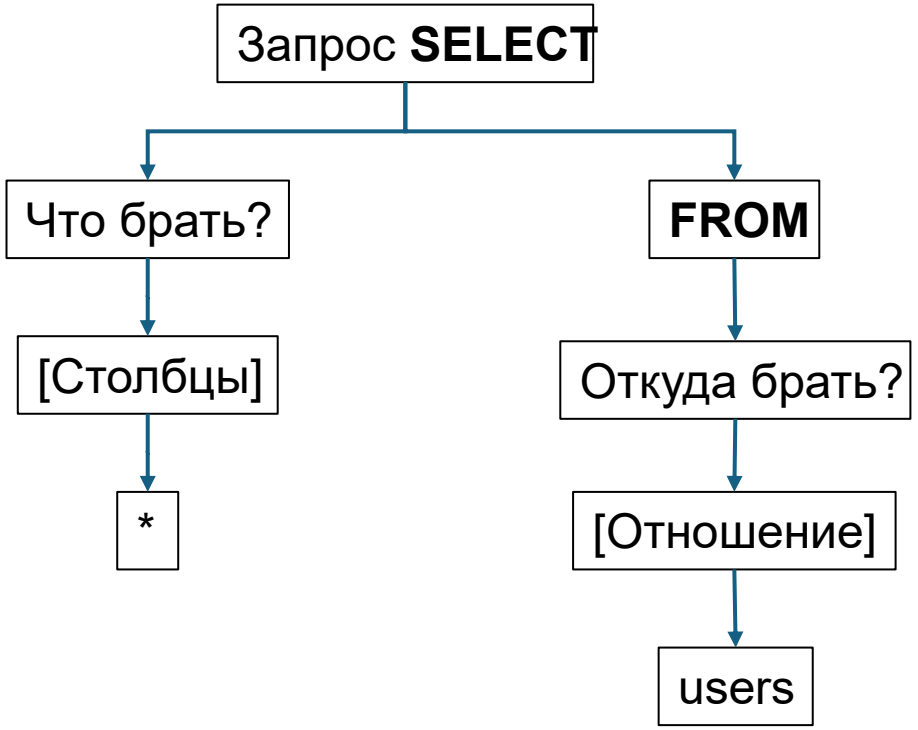
- На этом этапе устанавливается соединение между клиентом и СУБД
- В рамках соединения можно делать SQL-запросы



Жизненный цикл запроса

- Строка запроса разбивается на единицы информации (токены), которые будут понятны программе СУБД





Жизненный цикл запроса

- СУБД выполняет проверку, возможно ли выполнение запроса:
  - существует ли таблица users?
  - соблюдаются ли ограничения целостности?
  - есть ли права у пользователя на выполнение запросов к этой таблице?

Жизненный цикл запроса

- СУБД строит план того, как будет выполняться запрос:
  - выбирает наилучший алгоритм выполнения запроса из множества доступных в СУБД
  - определяет порядок выполнения операторов SQL

SELECT * **FROM** users;

1. Прочитать содержимое таблицы users (FROM users)
2. Сделать выборку всех столбцов и прочитанного (SELECT id, name, age)

Жизненный цикл запроса

- Движок СУБД выполняет запрос согласно плану

```
// sql.hpp
using namespace std;

// выбирает нужную проекцию из объекта Table
DbResult Select(
  const vector<string>& projection, const Table& table
);

// вычитывает данные из файла с названием table_name
Table From(const string& table_name);
```

1. Прочитать содержимое таблицы users (FROM users)
2. Сделать выборку всех столбцов и прочитанного (SELECT id, name, age)

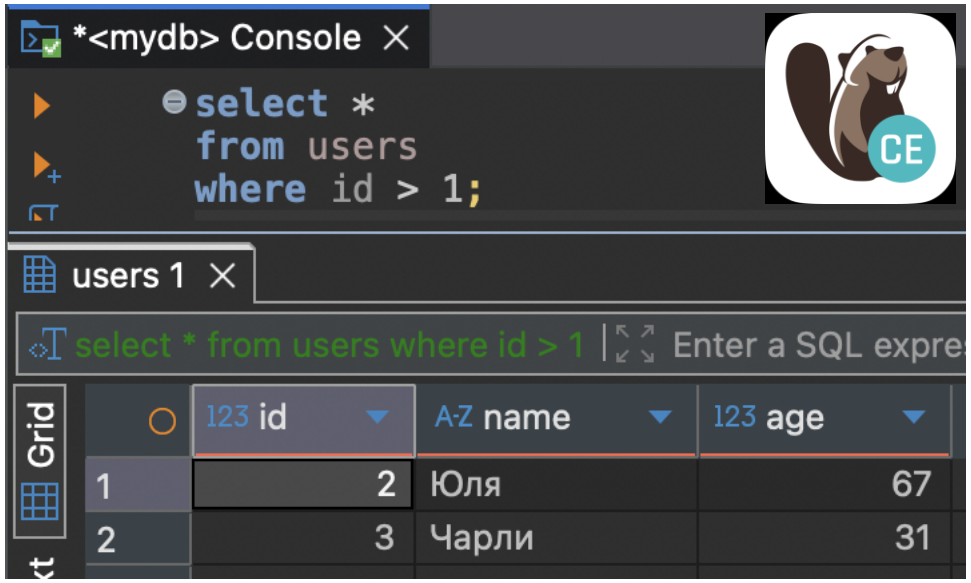
```
DbResult Execute() {
  Table tbl = From(”users”);
  return Select({“id”, “name”, “age”}, tbl);
}
```

Жизненный цикл запроса

Подключение → Парсинг → Проверка → Оптимизация → Выполнение → **Ответ**

- Полученный ответ возвращается клиенту

Выполнение запроса из клиента DBeaver:



План кейс-чемпионата

Цель и требования

- Цель – разработать свою собственную реляционную БД и СУБД к ней
- Работа в командах до 5 человек
- Требования:
  - данные хранятся в постоянном хранилище
  - взаимодействие клиента с СУБД вынесено в C++ библиотеку
  - используется хотя бы 1 паттерн проектирования
  - сборка и запуск осуществляются через скрипты, зависимости устанавливаются автоматически, полный процесс запуска описан

Минимально жизнеспособный продукт (MVP)

- Можно делать DDL-запросы в базовом виде:
  - **CREATE DATABASE** <имя_бд>;
  - **DROP DATABASE** <имя_бд>;
  - **CREATE TABLE** <имя_таблицы> …;
  - **DROP TABLE** <имя_таблицы>;
- Можно делать DML CRUD-запросы в базовом виде:
  - **SELECT** <проекция | *> **FROM** <имя_таблицы> [**WHERE** <условие>];
  - **INSERT INTO** <имя_таблицы> [(<список_столбцов>)] **VALUES** (<список_значений>) [, (<список_значений>)];
  - **UPDATE** <имя_таблицы> **SET** <столбец1> = <значение1> [, <столбецN> = <значениеN>] [**WHERE** <условие>];
  - **DELETE FROM** <имя_таблицы> [**WHERE** <условие>];
- Есть базовый интерфейс для запросов (консольный или графический):
  - запросы пользователь пишет текстом
  - результат запроса показан в интерфейсе (если выборка – видно таблицу, если изменение – видно количество измененных строк)

Материалы

- Копия презентации (DB_Pres.pdf)
- Полное описание требований и MVP (DB_MVP.md)
  - **При возможных разночтениях требований в презентации и документе ориентироваться на документ!**

https://disk.yandex.ru/d/1kmxSwvyYYkSIA

На дополнительные баллы (Post-MVP)

- Добавляем любую другую функциональность, которая относится к реляционным СУБД
- Идеи:
  - расширить набор операторов
  - добавить систему ролей, доступов
  - добавить красивый интерфейс
  - SQL-запросы обрабатываются парсером и лексером и преобразуются в AST-дерево
  - хорошая организация проекта для дальнейшего расширения функциональности
- Чем больше и сложнее, тем лучше

Отчет

- Предоставляются следующие материалы:
  - PDF-отчет
  - видео-запись демонстрации работы
  - архив с программой
- Что должно быть:
  - Список возможностей вашей СУБД
  - UML-диаграмма классов

Награждение победителей и участников

Этап 1. Ранжирование команд

По результатам выполненных заданий происходит ранжирование всех команд.

Затем формируются первые 5 мест.

Этап 2. Награждение за участие

Команды, занявшие 4 и 5 места, получают награды за участие в кейс-чемпионате.

Этап 3. Защита проектов

Команды, занявшие 1, 2 и 3 места, продолжают соревнование, защищая свои проекты перед жюри.

Этап 4. Определение призёров

По результатам защиты формируются три призовых места: 1, 2 и 3.